```
    assertEquals("ProxyTestName1", p.getName());
    assertEquals("ProxyTest1", p.getSku());
  }
}
```

In order to make this work, we have to separate the interface of `Product` from its implementation. So I changed `Product` to an interface and created `ProductImp` to implement it. (See Listings 26-11 and 26-12.)

Notice that I have added exceptions to the `Product` interface. This is because I was writing `ProductProxy` (Listing 26-13) at the same time that I was writing `Product`, `ProductImp`, and `ProxyTest`. I implemented them all one accessor at a time. As we will see, the `ProductProxy` class invokes the database, which throws exceptions. I did not want those exceptions to be caught and hidden by the proxy, so I decided to let them escape from the interface.

Listing 26-11

Product.java

```
public interface Product
{
  public int getPrice() throws Exception;
  public String getName() throws Exception;
  public String getSku() throws Exception;
}
```

Listing 26-12

ProductImp.java

```
public class ProductImp implements Product
{
  private int itsPrice;
  private String itsName;
  private String itsSku;

  public ProductImp(String sku, String name, int price)
  {
    itsPrice = price;
    itsName = name;
    itsSku = sku;
  }

  public int getPrice()
  {
    return itsPrice;
  }

  public String getName()
  {
    return itsName;
  }

  public String getSku()
  {
    return itsSku;
  }
}
```

Listing 26-13

ProductProxy.java

```
public class ProductProxy implements Product
{
  private String itsSku;
  public ProductProxy(String sku)
  {
    itsSku = sku;
  }
  public int getPrice() throws Exception
  {
    ProductData pd = DB.getProductData(itsSku);
    return pd.price;
  }

  public String getName() throws Exception
  {
    ProductData pd = DB.getProductData(itsSku);
    return pd.name;
  }

  public String getSku() throws Exception
  {
    return itsSku;
  }
}
```

The implementation of this proxy is trivial. In fact, it doesn't quite match the canonical form of the pattern shown in Figures 26-3 and 26-4. This was an unexpected surprise. My intent was to implement the PROXY pattern. But when the implementation finally materialized, the canonical pattern made no sense.

As shown below, the canonical pattern would have had `ProductProxy` create a `ProductImp` in every method. It would then have delegated that method to the `ProductImp`.

```
public int getPrice() throws Exception
{
  ProductData pd = DB.getProductData(itsSku);
  ProductImp p = new ProductImp(pd.sku, pd.name, pd.price);
  return p.getPrice();
}
```

The creation of the `ProductImp` is a complete waste of programmer and computer resources. The `ProductProxy` already has the data that the `ProductImp` accessors would return. So there is no need to create, and then delegate to, the `ProductImp`. This is yet another example of how the code may lead you away from the patterns and models you expected.

Notice that the `getSku` method of `ProductProxy` in Listing 26-13 takes this theme one step further. It doesn't even bother to hit the database for the `sku`. Why should it? It already has the `sku`.

You might be thinking that the implementation of `ProductProxy` is very inefficient. It hits the database for each accessor. Wouldn't it be better if it cached the `ProductData` item in order to avoid hitting the database?

This change is trivial, but the only thing driving us to do it is our fear. At this point, we have no data to suggest that this program has a performance problem. And besides, we know that the database engine is doing some caching too. So it's not clear what building our own cache would buy us. We should wait until we see indications of a performance problem before we invent trouble for ourselves.

Proxyifying Relationships. Our next step is to create the proxy for `Order`. Each `Order` instance contains many `Item` instances. In the relational schema (Figure 26-2), this relationship is captured within the `Item` table. Each row of the `Item` table contains the key of the `Order` that contains it. In the object model, however, the relationship is implemented by a `Vector` within `Order`. (See Listing 26-4.) Somehow the proxy is going to have to translate between the two forms.

We begin by posing a test case that the proxy must pass. This test adds a few dummy products to the database. It then obtains proxies to those products, and uses them to invoke `addItem` on an `OrderProxy`. Finally, it asks the `OrderProxy` for the total price. (See Listing 26-14.) The intent of this test case is to show that an `OrderProxy` behaves just like an `Order`, but that it obtains its data from the database instead of from in-memory objects.

Listing 26-14

ProxyTest.java

```
public void testOrderProxyTotal() throws Exception
  {
    DB.store(new ProductData("Wheaties", 349, "wheaties"));
    DB.store(new ProductData("Crest", 258, "crest"));
    ProductProxy wheaties = new ProductProxy("wheaties");
    ProductProxy crest = new ProductProxy("crest");
    OrderData od = DB.newOrder("testOrderProxy");
    OrderProxy order = new OrderProxy(od.orderId);
    order.addItem(crest, 1);
    order.addItem(wheaties, 2);
    assertEquals(956, order.total());
  }
```

In order to make this test case work, we have to implement a few new classes and methods. The first we'll tackle is the `newOrder` method of `DB`. It looks like this method returns an instance of something called an `OrderData`. `OrderData` is just like `ProductData`. It is a simple data structure that represents a row of the `Order` database table. It is shown in Listing 26-15.

Listing 26-15

OrderData.java

```
public class OrderData
{
  public String customerId;
  public int orderId;

  public OrderData()
  {
  }

  public OrderData(int orderId, String customerId)
  {
    this.orderId = orderId;
    this.customerId = customerId;
  }
}
```

Don't be offended by the use of public data members. This is not an object in the true sense. It is just a container for data. It has no interesting behavior that needs to be encapsulated. Making the data variables private and providing getters and setters would just be a needless complication.

Now we need to write the `newOrder` function of `DB`. Notice that when we call it in Listing 26-14, we provide the ID of the owning customer, but we do not provide the `orderId`. Each `Order` needs an `orderId` to act as its key. What's more, in the relational schema, each `Item` refers to this `orderId` as a way to show its connection to the `Order`. Clearly the `orderId` must be unique. How does it get created? Let's write a test to show our intent. (See Listing 26-16.)

Listing 26-16

DBTest.java

```
public void testOrderKeyGeneration() throws Exception
{
  OrderData o1 = DB.newOrder("Bob");
  OrderData o2 = DB.newOrder("Bill");
  int firstOrderId = o1.orderId;
  int secondOrderId = o2.orderId;
  assertEquals(firstOrderId+1, secondOrderId);
}
```

This test shows that we expect the `orderId` to somehow automatically increment every time a new `Order` is created. This is easily implemented by querying the database for the maximum `orderId` currently in use and then adding one to it. (See Listing 26-17.)

Listing 26-17

DB.java

```
  public static OrderData newOrder(String customerId) throws Exception
  {
    int newMaxOrderId = getMaxOrderId() + 1;
    PreparedStatement s = con.prepareStatement(
      "Insert into Orders(orderId,cusid) Values(?,?);");
    s.setInt(1, newMaxOrderId);
    s.setString(2,customerId);
    executeStatement(s);
    return new OrderData(newMaxOrderId, customerId);
  }

  private static int getMaxOrderId() throws SQLException
  {
    Statement qs = con.createStatement();
    ResultSet rs = qs.executeQuery(
      "Select max(orderId) from Orders;");
    rs.next();
    int maxOrderId = rs.getInt(1);
    rs.close();
    return maxOrderId;
  }
```

Now we can start to write `OrderProxy`. As with `Product`, we need to split `Order` into an interface and an implementation. So `Order` becomes the interface and `OrderImp` becomes the implementation. (See Listings 26-18 and 26-19.)

Listing 26-18

Order.java

```
public interface Order
{
  public String getCustomerId();
  public void addItem(Product p, int quantity);
  public int total();
}
```

Listing 26-19

OrderImp.java

```
import java.util.Vector;

public class OrderImp implements Order
{
  private Vector itsItems = new Vector();
  private String itsCustomerId;

  public String getCustomerId()
  {
    return itsCustomerId;
  }

  public OrderImp(String cusid)
  {
    itsCustomerId = cusid;
  }

  public void addItem(Product p, int qty)
  {
    Item item = new Item(p,qty);
    itsItems.add(item);
  }

  public int total()
  {
    try
    {
      int total = 0;
      for (int i = 0; i < itsItems.size(); i++)
      {
        Item item = (Item) itsItems.elementAt(i);
        Product p = item.getProduct();
        int qty = item.getQuantity();
        total += p.getPrice() * qty;
      }
      return total;
    }
```

```
    catch (Exception e)
    {
      throw new Error(e.toString());
    }
  }
}
```

I had to add some exception processing to `OrderImp` because the `Product` interface throws exceptions. I'm getting frustrated with all these exceptions. The implementations of proxies behind an interface should not have an effect on that interface, and yet the proxies are throwing exceptions that propagate out through the interface. So I resolve to change all the `Exceptions` to `Errors` so that I don't have to pollute the interfaces with `throws` clauses and the users of those interfaces with `try/catch` blocks.

How do I implement `addItem` in the proxy? Clearly the proxy cannot delegate to `OrderImp.addItem`! Rather, the proxy is going to have to insert an `Item` row in the database. On the other hand, I *really want* to delegate `OrderProxy.total` to `OrderImp.total`, because I want the business rules (i.e., the policy for creating totals) to be encapsulated in `OrderImp`. The whole point of building proxies is to separate database implementation from business rules.

In order to delegate the `total` function, the proxy is going to have to build the complete `Order` object along with all its contained `Items`. Thus, in `OrderProxy.total`, we are going to have to read in all the `items` from the database, call `addItem` on an empty `OrderImp` for each `item` we find, and then call `total` on that `OrderImp`. Thus, the `OrderProxy` implementation ought to look something like Listing 26-20.

Listing 26-20

`OrderProxy.java`

```
import java.sql.SQLException;

public class OrderProxy implements Order
{
  private int orderId;

  public OrderProxy(int orderId)
  {
    this.orderId = orderId;
  }

  public int total()
  {
    try
    {
      OrderImp imp = new OrderImp(getCustomerId());
      ItemData[] itemDataArray = DB.getItemsForOrder(orderId);
      for (int i = 0; i < itemDataArray.length; i++)
      {
        ItemData item = itemDataArray[i];
        imp.addItem(new ProductProxy(item.sku), item.qty);
      }
      return imp.total();
    }
    catch (Exception e)
    {
      throw new Error(e.toString());
    }
  }
```

```
  public String getCustomerId()
  {
    try
    {
      OrderData od = DB.getOrderData(orderId);
      return od.customerId;
    }
    catch (SQLException e)
    {
      throw new Error(e.toString());
    }
  }

  public void addItem(Product p, int quantity)
  {
    try
    {
      ItemData id =
        new ItemData(orderId, quantity, p.getSku());
      DB.store(id);
    }
    catch (Exception e)
    {
      throw new Error(e.toString());
    }
  }

  public int getOrderId()
  {
    return orderId;
  }
}
```

This implies the existence of an `ItemData` class and a few `DB` functions for manipulating `ItemData` rows. These are shown in Listings 26-21 through 26-23.

Listing 26-21

ItemData.java

```
public class ItemData
{
  public int orderId;
  public int qty;
  public String sku = "junk";

  public ItemData()
  {
  }

  public ItemData(int orderId, int qty, String sku)
  {
    this.orderId = orderId;
    this.qty = qty;
    this.sku = sku;
  }
```

```
  public boolean equals(Object o)
  {
    ItemData id = (ItemData)o;
    return orderId == id.orderId &&
           qty == id.qty &&
           sku.equals(id.sku);
  }
}
```

Listing 26-22

DBTest.java

```
  public void testStoreItem() throws Exception
  {
    ItemData storedItem = new ItemData(1, 3, "sku");
    DB.store(storedItem);
    ItemData[] retrievedItems = DB.getItemsForOrder(1);
    assertEquals(1, retrievedItems.length);
    assertEquals(storedItem, retrievedItems[0]);
  }

  public void testNoItems() throws Exception
  {
    ItemData[] id = DB.getItemsForOrder(42);
    assertEquals(0, id.length);
  }
```

Listing 26-23

DB.java

```
  public static void store(ItemData id) throws Exception
  {
    PreparedStatement s = buildItemInsersionStatement(id);
    executeStatement(s);
  }

  private static PreparedStatement
  buildItemInsersionStatement(ItemData id) throws SQLException
  {
    PreparedStatement s = con.prepareStatement(
      "Insert into Items(orderId,quantity,sku) " +
      "VALUES (?, ?, ?);");
    s.setInt(1,id.orderId);
    s.setInt(2,id.qty);
    s.setString(3, id.sku);
    return s;
  }

  public static ItemData[] getItemsForOrder(int orderId)
  throws Exception
  {
    PreparedStatement s =
      buildItemsForOrderQueryStatement(orderId);
    ResultSet rs = s.executeQuery();
    ItemData[] id = extractItemDataFromResultSet(rs);
    rs.close();
```

```
    s.close();
    return id;
  }

  private static PreparedStatement
  buildItemsForOrderQueryStatement(int orderId)
  throws SQLException
  {
    PreparedStatement s = con.prepareStatement(
      "SELECT * FROM Items WHERE orderid = ?;");
    s.setInt(1, orderId);
    return s;
  }

  private static ItemData[]   extractItemDataFromResultSet(ResultSet rs)
  throws SQLException
  {
    LinkedList l = new LinkedList();
    for (int row = 0; rs.next(); row++)
    {
      ItemData id = new ItemData();
      id.orderId = rs.getInt("orderid");
      id.qty = rs.getInt("quantity");
      id.sku = rs.getString("sku");
      l.add(id);
    }
    return (ItemData[]) l.toArray(new ItemData[l.size()]);
  }

  public static OrderData getOrderData(int orderId)
  throws SQLException
  {
    PreparedStatement s = con.prepareStatement(
      "Select cusid from orders where orderid = ?;");
    s.setInt(1, orderId);
    ResultSet rs = s.executeQuery();
    OrderData od = null;
    if (rs.next())
      od =  new OrderData(orderId, rs.getString("cusid"));
    rs.close();
    s.close();
    return od;
  }
```

Summary of PROXY

This example should have dispelled any false illusions about the elegance and simplicity of using proxies. Proxies are not trivial to use. The simple delegation model implied by the canonical pattern seldom materializes so neatly. Rather, we find ourselves short-circuiting the delegation for trivial getters and setters. For methods that manage `1:N` relationships, we find ourselves *delaying* the delegation and moving it into other methods, just as the delegation for `addItem` was moved into `total`. Finally, we face the spectre of caching.

We didn't do any caching in this example. The tests all run in less than a second, so there was no need to worry overmuch about performance. But in a real application, the issue of performance and the need for intelligent caching are likely to arise. I do not suggest that you automatically implement a caching strategy because you fear

performance will otherwise be too slow. Indeed, I have found that adding caching too early is a very good way to *decrease* performance. If you fear performance may be a problem, I recommend that you conduct some experiments to *prove* that it will be a problem. Once proven, and *only once proven*, you should start considering how to speed things up.

The Benefit of PROXY. For all the troublesome nature of proxies, they have one very powerful benefit: *the separation of concerns*. In our example, the business rules and the database have been completely separated. `OrderImp` has no dependence whatever on the database. If we want to change the database schema or change the database engine, we can do so without affecting `Order`, `OrderImp`, or any of the other business domain classes.

In those instances where separation of business rules from database implementation is critically important, PROXY can be a good pattern to employ. For that matter, PROXY can be used to separate business rules from *any* kind of implementation issue. It can be used to keep the business rules from being polluted by such things as COM, CORBA, EJB, etc. It is a way to keep the business rule assets of your project separate from the implementation mechanisms that are currently in vogue.

Dealing with Databases, Middleware, and Other Third Party Interfaces

Third party APIs are a fact of life for software engineers. We buy database engines, middleware engines, class libraries, threading libraries, etc. Initially, we use these APIs by making direct calls to them from our application code. (See Figure 26-5.)

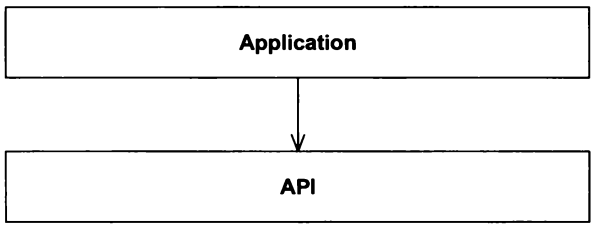


Figure 26-5 Initial relationship between an application and a third party API

Over time, however, we find that our application code becomes more and more polluted with such API calls. In a database application, for example, we may find more and more SQL strings littering the code that also contains the business rules.

This becomes a problem when the third party API changes. For databases it also becomes a problem when the schema changes. As new versions of the API or schema are released, more and more of the application code has to be reworked to align with those changes.

Eventually, the developers decide that they must insulate themselves from these changes. So they invent a layer that separates the application business rules from the third party API. (See Figure 26-6.) They concentrate into this layer all the code that uses the third party API and all of the concepts that are related to the API rather than to the business rules of the application.

Such layers can sometimes be purchased. ODBC or JDBC are such layers. They separate the application code from the actual database engine. Of course, they are also third party APIs in and of themselves; therefore, the application may need to be insulated even from them.

Notice that there is a transitive dependency from the `Application` to the `API`. In some applications, that indirect dependence is still enough to cause problems. JDBC, for example, does not insulate the application from the details of the schema.

In order to attain even better insulation, we need to invert the dependency between the application and the layer. (See Figure 26-7.) This keeps the application from knowing anything at all about the third party API, either directly or indirectly. In the case of a database, it keeps the application from direct knowledge of the schema. In the

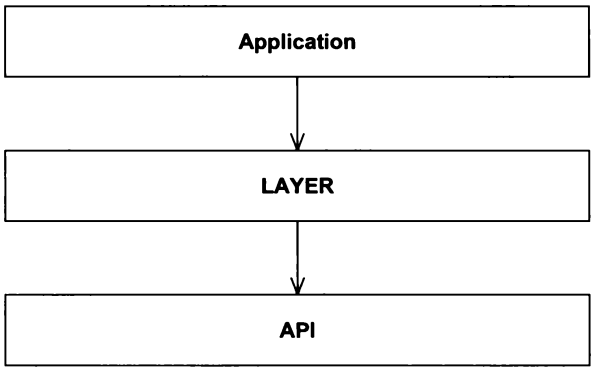


Figure 26-6 Introducing an insulation layer

case of a middleware engine, it keeps the application from knowing anything about the datatypes used by that middleware processor.

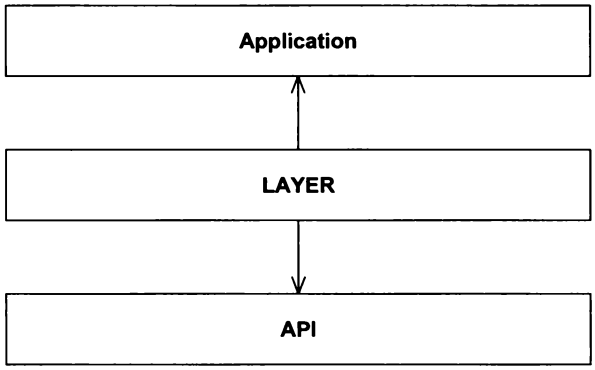


Figure 26-7 Inverting the dependency between the application and Layer

This arrangement of dependencies is precisely what the PROXY pattern achieves. The application does not depend upon the proxies at all. Rather, the proxies depend upon the application and upon the API. This concentrates all knowledge of the mapping between the application and the API into the proxies.

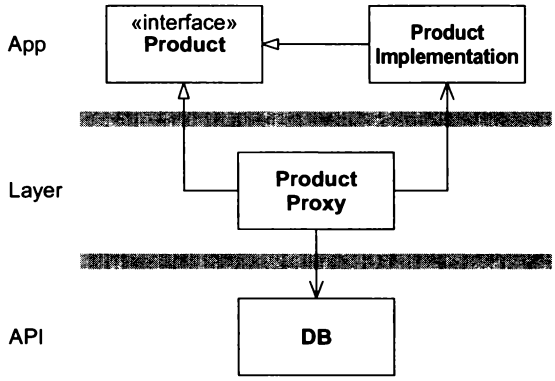


Figure 26-8 How the PROXY inverts the dependency between the application and the Layer

This concentration of knowledge means that the proxies are nightmares. Whenever the API changes, the proxies change. Whenever the application changes, the proxies change. The proxies can become very hard to deal with.

It's good to know where your nightmares live. Without the proxies, the nightmares would be spread throughout the application code.

Most applications don't need proxies. Proxies are a very heavyweight solution. When I see proxy solutions in use, my recommendation in most cases is to take them out and use something simpler. But there are cases when the intense separation between the application and the API afforded by proxies is beneficial. Those cases are almost always in very large systems that undergo frequent schema or API thrashing. Or they are in systems that can ride on top of many different database engines or middleware engines.

STAIRWAY TO HEAVEN[1]

STAIRWAY TO HEAVEN is another pattern that achieves the same dependency inversion as PROXY. It employs a variation on the class form of the ADAPTER pattern. (See Figure 26-9.)

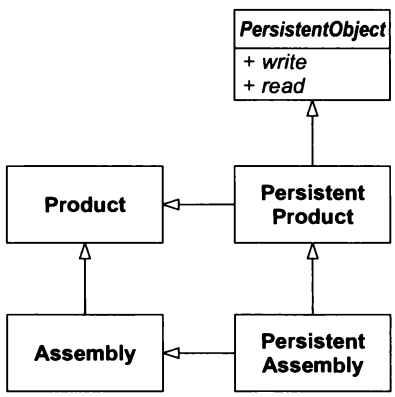


FIGURE 26-9 STAIRWAY TO HEAVEN

`PersistentObject` is an abstract class that knows about the database. It provides two abstract methods: `read` and `write`. It also provides a set of implemented methods that provides the tools needed to implement `read` and `write`. `PersistentProduct`, for example, uses these tools to implement `read` and `write` to read and write all the data fields of `Product` from and to the database. By the same token, `PersistentAssembly` implements `read` and `write` to do the same for the extra fields within `Assembly`. It inherits the ability to read and write the fields of `Product` from `PersistentProduct` and structures the `read` and `write` methods so as to take advantage of that fact.

This pattern is only useful in languages that support multiple inheritance. Note that both `PersistentProduct` and `PersistentAssembly` inherit from two implemented base classes. What's more, `PersistentAssembly` finds itself in a *diamond* inheritance relationship with `Product`. In C++, we use virtual inheritance to prevent two instances of `Product` from being inherited into `PersistentAssembly`.

The need for virtual inheritance, or similar relationships in other languages, means that this pattern is somewhat intrusive. It makes itself felt in the `Product` hierarchy, but the intrusion is minimal.

The benefit of this pattern is that it completely separates knowledge of the database away from the business rules of the application. Those small bits of the application that need to invoke `read` and `write` can do so through the following exigency:

1. [Martin97].

```
PersistentObject* o = dynamic_cast<PersistentObject*>(product);
if (o)
  o->write();
```

In other words, we ask the application object if it conforms to the `PersistentObject` interface, and if so, we invoke either `read` or `write`. This keeps that part of the application that does not need to know about reading and writing completely independent of the `PersistentObject` side of the hierarchy.

Example of STAIRWAY TO HEAVEN

Listings 26-24 through 26-34 show an example of STAIRWAY TO HEAVEN in C++. As usual, it is best to start with the test case. CppUnit[2] is a bit wordy if shown in its entirety, so I have only included the test-case methods in Listing 26-24. The first test-case verifies that a `PersistentProduct` can be passed around the system as a `Product` and then converted to a `PersistentObject` and written at will. We assume that the `PersistentProduct` will write itself in a simple XML format. The second test case verifies the same for `PersistentAssembly`, the only difference being the addition of a second field in the `Assembly` object.

Listing 26-24

productPersistenceTestCase.cpp {abridged}

```
void ProductPersistenceTestCase::testWriteProduct()
{
  ostrstream s;
  Product* p = new PersistentProduct("Cheerios");
  PersistentObject* po = dynamic_cast<PersistentObject*>(p);
  assert(po);
  po->write(s);
  char* writtenString = s.str();
  assert(strcmp("<PRODUCT><NAME>Cheerios</NAME></PRODUCT>",
                writtenString) == 0);
}

void ProductPersistenceTestCase::testWriteAssembly()
{
  ostrstream s;
  Assembly* a = new PersistentAssembly("Wheaties", "7734");
  PersistentObject* po = dynamic_cast<PersistentObject*>(a);
  assert(po);
  po->write(s);
  char* writtenString = s.str();
  assert(strcmp("<ASSEMBLY><NAME>Wheaties"
                "</NAME><ASSYCODE>7734</ASSYCODE></ASSEMBLY>",
                 writtenString) == 0);

}
```

Next, in Listings 26-25 through 26-28, we see the definitions and implementations of both `Product` and `Assembly`. In the interest of saving space in our example, these classes are nearly degenerate. In a normal application, these classes would contain methods that implemented business rules. Note that there is no hint of persistence in either of these classes. There is no dependence whatever from the business rules to the persistence mechanism. This is the whole point of the pattern.

2. One of the XUnit family of unit test frameworks. See `www.junit.org`, and `www.xprogramming.com` for more information.

While the dependency characteristics are good, there is an artifact in Listing 26-27 that is present solely because of the STAIRWAY TO HEAVEN pattern. `Assembly` inherits from `Product` using the `virtual` keyword. This is necessary in order to prevent duplicate inheritance of `Product` in `PersistentAssembly`. If you refer back to Figure 26-9, you'll see that `Product` is the apex of a diamond[3] of inheritance involving `Assembly`, `PersistentProduct`, and `PersistentObject`. To prevent duplicate inheritance of `Product`, it must be inherited virtually.

Listing 26-25

product.h

```
#ifndef STAIRWAYTOHEAVENPRODUCT_H
#define STAIRWAYTOHEAVENPRODUCT_H

#include <string>

class Product
{
 public:
  Product(const string& name);
  virtual ~Product();
  const string& getName() const {return itsName;}
 private:
  string itsName;
};

#endif
```

Listing 26-26

product.cpp

```
#include "product.h"

Product::Product(const string& name)
  : itsName(name)
{
}

Product::~Product()
{
}
```

Listing 26-27

assembly.h

```
#ifndef STAIRWAYTOHEAVENASSEMBLY_H
#define STAIRWAYTOHEAVENASSEMBLY_H

#include <string>
#include "product.h"

class Assembly : public virtual Product
{
 public:
```

3. Sometimes facetiously known as the "deadly diamond of death."

```
  Assembly(const string& name, const string& assyCode);
  virtual ~Assembly();

  const string& getAssyCode() const {return itsAssyCode;}
 private:
  string itsAssyCode;
};

#endif
```

Listing 26-28

assembly.cpp

```
#include "assembly.h"

Assembly::Assembly(const string& name, const string& assyCode)
  :Product(name), itsAssyCode(assyCode)
{
}

Assembly::~Assembly()
{
}
```

Listings 26-29 and 26-30 show the definition and implementation of `PersistentObject`. Note that while `PersistentObject` knows nothing of the `Product` hierarchy, it does seem to know something about how to write XML. At least it understands that objects are written by writing a header, followed by the fields, followed by a footer.

The `write` method of `PersistentObject` uses the TEMPLATE METHOD[4] pattern to control the writing of all its derivatives. Thus, the persistent side of the STAIRWAY TO HEAVEN pattern makes use of the facilities of the `PersistentObject` base class.

Listing 26-29

persistentObject.h

```
#ifndef STAIRWAYTOHEAVENPERSISTENTOBJECT_H
#define STAIRWAYTOHEAVENPERSISTENTOBJECT_H

#include <iostream>

class PersistentObject
{
 public:
  virtual ~PersistentObject();
  virtual void write(ostream&) const;

 protected:
  virtual void writeFields(ostream&) const = 0;

 private:
  virtual void writeHeader(ostream&) const = 0;
  virtual void writeFooter(ostream&) const = 0;
};

#endif
```

4. See Chapter 14: *Template Method & Strategy: Inheritance vs. Delegation*, on page 161.

Listing 26-30

persistentObject.cpp

```
#include "persistentObject.h"

PersistentObject::~PersistentObject()
{
}

void PersistentObject::write(ostream& s) const
{
  writeHeader(s);
  writeFields(s);
  writeFooter(s);
  s << ends;
}
```

Listings 26-31 and 26-32 show the implementation of `PersistentProduct`. This class implements the `writeHeader`, `writeFooter`, and `writeField` functions to create the appropriate XML for a `Product`. It inherits the fields and accessors from `Product` and is driven by the write method of its base class `PersistentObject`.

Listing 26-31

persistentProduct.h

```
#ifndef STAIRWAYTOHEAVENPERSISTENTPRODUCT_H
#define STAIRWAYTOHEAVENPERSISTENTPRODUCT_H

#include "product.h"
#include "persistentObject.h"

class PersistentProduct : public virtual Product
                        , public PersistentObject
{
 public:
  PersistentProduct(const string& name);
  virtual ~PersistentProduct();

 protected:
  virtual void writeFields(ostream& s) const;

 private:
  virtual void writeHeader(ostream& s) const;
  virtual void writeFooter(ostream& s) const;
};

#endif
```

Listing 26-32

persistentProduct.cpp

```
#include "persistentProduct.h"

PersistentProduct::PersistentProduct(const string& name)
:Product(name)
{
}
```

```
PersistentProduct::~PersistentProduct()
{
}

void PersistentProduct::writeHeader(ostream& s) const
{
  s << "<PRODUCT>";
}

void PersistentProduct::writeFooter(ostream& s) const
{
  s << "</PRODUCT>";
}

void PersistentProduct::writeFields(ostream& s) const
{
  s << "<NAME>" << getName() << "</NAME>";
}
```

Finally, Listings 26-33 and 26-34 show how `PersistentAssembly` unifies `Assembly` and `PersistentProduct`. Just like `PersistentProduct`, it overrides `writeHeader`, `writeFooter`, and `writeFields`. However, it implements `writeFields` to invoke `PersistentProduct::writeFields`. Thus, it inherits the ability to write the `Product` part of `Assembly` from `PersistentProduct`, and it inherits the `Product` and `Assembly` fields and accessors from `Assembly`.

Listing 26-33

persistentAssembly.h

```
#ifndef STAIRWAYTOHEAVENPERSISTENTASSEMBLY_H
#define STAIRWAYTOHEAVENPERSISTENTASSEMBLY_H

#include "assembly.h"
#include "persistentProduct.h"

class PersistentAssembly : public Assembly, public PersistentProduct
{
 public:
  PersistentAssembly(const string& name,
                     const string& assyCode);
  virtual ~PersistentAssembly();

 protected:
  virtual void writeFields(ostream& s) const;

 private:
  virtual void writeHeader(ostream& s) const;
  virtual void writeFooter(ostream& s) const;
};

#endif
```

Listing 26-34

persistentAssembly.cpp

```
#include "persistentAssembly.h"

PersistentAssembly::PersistentAssembly(const string& name, const string& assyCode)
: Assembly(name, assyCode)
, PersistentProduct(name)
, Product(name)
{
}

PersistentAssembly::~PersistentAssembly()
{
}

void PersistentAssembly::writeHeader(ostream& s) const
{
  s << "<ASSEMBLY>";
}

void PersistentAssembly::writeFooter(ostream& s) const
{
  s << "</ASSEMBLY>";
}

void PersistentAssembly::writeFields(ostream& s) const
{
  PersistentProduct::writeFields(s);
  s << "<ASSYCODE>" << getAssyCode() << "</ASSYCODE>";
}
```

Conclusion. I've seen STAIRWAY TO HEAVEN used in many different scenarios with good results. The pattern is relatively easy to set up and has a minimum impact on the objects that contain the business rules. On the other hand, it requires a language, like C++, that supports multiple inheritance of implementation.

Other Patterns That Can Be Used with Databases

Extension Object. Imagine an extension object[5] that knows how to write the extended object on a database. In order to write such an object, you would ask it for an extension object that matched the "database" key, cast it to a `DatabaseWriterExtension`, and then invoke the `write` function.

```
Product p = /* some function that returns a Product */
ExtensionObject e = p.getExtension("Database");
if (e != null)
{
  DatabaseWriterExtension dwe = (DatabaseWriterExtension) e;
  e.write();
}
```

Visitor[6]. Imagine a visitor hierarchy that knows how to write the visited object on a database. You would write an object on the database by creating the appropriate type of visitor, and then calling `accept` on the object to be written.

5. See "Extension Object" on page 408
6. See "Visitor" on page 388

```
Product p = /* some function that returns a Product */
DatabaseWriterVisitor dwv = new DatabaseWriterVisitor();
p.accept(dwv);
```

Decorator[7]**.** There are two ways to use a decorator to implement databases. You can decorate a business object and give it `read` and `write` methods; or you can decorate a data object that knows how to read and write itself and give it business rules. The latter approach is not uncommon when using object-oriented databases. The business rules are kept out of the OODB schema and added in with decorators.

Facade. This is my favorite starting point. It's simple and effective. On the down side, it couples the business-rule objects with the database. Figure 26-10 shows the structure. The `DatabaseFacade` class simply provides methods for reading and writing all the necessary objects. This couples the objects with the `DatabaseFacade` and vice versa. The objects know about the facade because they are often the ones that call the `read` and `write` functions. The facade knows about the objects because it must use their accessors and mutators to implement the `read` and `write` functions.

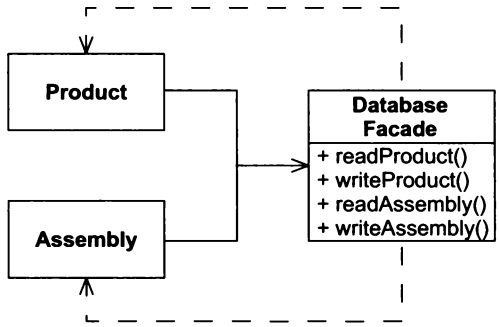


Figure 26-10 Database Facade

This coupling can cause a lot of problems in larger applications; but in smaller apps or in apps that are just starting to grow, it's a pretty effective technique. If you start using a facade and then later decide to change to one of the other patterns to reduce coupling, the facade is pretty easy to refactor.

Conclusion

It is very tempting to anticipate the need for PROXY or STAIRWAY TO HEAVEN long before the need really exists. This is almost never a good idea, especially with PROXY. I recommend starting with FACADE and then refactoring as necessary. You'll save yourself time and trouble if you do.

Bibliography

1. Gamma, et al. *Design Patterns*. Reading, MA: Addison–Wesley, 1995.
2. Martin, Robert C. Design Patterns for Dealing with Dual Inheritance Hierarchies. *C++ Report* (April): 1997.

7. See "Decorator" on page 403

27

Case Study: Weather Station

Written in collaboration with Jim Newkirk



The Following Story is fiction, but you may recognize many elements of it from your own exeperience.



The Cloud Company

The Cloud Company has been the leader in industrial weather monitoring systems (WMS) for the past several years. Their flagship product has been a WMS that keeps track of temperature, humidity, barometric pressure, wind speed and direction, etc. The system displays these readings in real time on a display. It also keeps track of historical information on an hourly and daily basis. This historical data can be pulled up on the display at the request of the user.

The primary customers of Cloud Company products have been the aviation, maritime, agricultural, and broadcast industries. For these industries, WMSs are mission-critical applications. The Cloud Company has a reputation for building highly reliable products that can be installed in relatively uncontrolled environments. This makes the systems somewhat expensive.

The high cost of these systems has cut the Cloud Company off from customers that do not need, and cannot afford, the high-reliability systems that they sell. Cloud Company managers believe that this is a large potential market, and they would like to tap into it.

The Problem. A competitor named Microburst, Inc., has recently announced a product line that starts at the low end and can be incrementally upgraded to higher reliability. This threatens to cut the Cloud Company off from smaller but growing customers. These customers will already be using Microburst products by the time they grow to a size that would allow them to use Cloud Company products.

More frightening still, the Microburst product boasts the ability to be interconnected at the high end. That is, the high-end upgrades can be networked together into a wide-area weather monitoring system. This threatens to erode the current Cloud Company customer base.

The Strategy. Although Microburst has successfully demonstrated its low-end units at trade shows, they are not offering production quantity shipments for at least six months. This indicates that there may be engineering or production problems that Microburst has not solved. Moreover, the high-reliability upgrades promised by Microburst as part of the product line are currently not available. It seems that Microburst has announced the product prematurely.

If the Cloud Company can announce a low-end upgradable and connectable product, and begin shipping it *within* six months, then they may be able to capture, or at least stall, customers who would otherwise buy Microburst's products. By stalling the market and thereby depriving Microburst of orders, they might be able to compromise Microburst's ability to solve their engineering and manufacturing problems, a very desirable outcome.

The Dilemma. A new low-cost and extendable product line requires a significant amount of engineering. The hardware engineers have flatly refused to commit to a six-month development deadline. They believe that it will be twelve months before they could see production-quantity units.

The marketing managers believe that in twelve months, Microburst will be shipping production quantity and will be capturing an irretrievable part of Cloud Company's customers.

The Plan. Cloud Company managers have decided to announce their new product line immediately and to begin accepting orders that will be shipped before six months have elapsed. They have named the new product Nimbus-LC 1.0. Their plan is to repackage the old, expensive, high-reliability hardware into a new enclosure with a nice LCD touch panel. The high manufacturing cost of these units means that the company will actually lose money on each one that they sell.

Concurrently, the hardware engineers will begin to develop the true low-cost hardware, which will be available in 12 months. This configuration of the product has been called Nimbus-LC 2.0. When production quantities are available, the Nimbus-LC 1.0 will be phased out.

When a Nimbus-LC 1.0 customer wants to upgrade to a higher level of service, his unit will be replaced with a Nimbus-LC 2.0 at no additional cost. Thus, the company is willing to lose money on this product for six months in order to capture, or at least stall, potential Microburst customers.

The WMS-LC Software

The software project for the Nimbus-LC project is complex. The developers must create a software product that can use both the existing hardware as well as the low-cost 2.0 hardware. Prototype units of the 2.0 hardware will not be available for nine months. Moreover, the processor on the 2.0 board is not likely to be the same as the processor on the 1.0 board. Still, the system must operate identically, regardless of which hardware platform it uses.

The hardware engineers will be writing the lowest-level hardware drivers, and they need the application software engineers to design the API for these drivers. This API must be available to the hardware engineers within the next four months. The software must be production ready in 6 months and must be working with the 2.0 hardware in 12 months. They want at least 6 weeks of Q/A for the 1.0 device, so the software engineers really have only 20 weeks to get the software working. Since the hardware platform for the 2.0 version is new, they need 8 to 10 weeks of Q/A. This eats up most of the 3-month period between first prototype and final shipment. Thus the software engineers will have very little time to make the new hardware work.

Software Planning Documents. The developers and marketing folks have written several documents that describe the Nimbus-LC project:

1. "Nimbus-LC Requirements Overview" on page 379
   This document describes the operating requirements of the Nimbus-LC system as they were understood at the time the project was begun.[1]
2. "Nimbus-LC Use Cases" on page 380
   This document describes the actors and use cases derived from the requirements document.
3. "Nimbus-LC Release Plan" on page 381
   This document describes the release plan for the software. This plan tries to address the major risks early in the project life cycle, while assuring that the software will be complete by the necessary deadlines.

Language Selection

The most important constraint upon the language is portability. The short development time, and the even shorter contact that the software engineers will have with the 2.0 hardware demand that both the 1.0 and 2.0 versions use the same software. That is, the source code needs to be identical, or nearly so. If the portability constraint cannot be met by the language, the release of the 2.0 version at the 12-month mark will be in severe jeopardy.

Fortunately, there are few other constraints. The software is not very large, so space is not much of a problem. There are no hard real-time deadlines that are shorter than one second, so speed is not much of an issue. Indeed, the real-time deadlines are so weak that a moderately fast garbage-collecting language would not be inappropriate. The portability constraints, and the lack of any other serious constraints, make the selection of Java quite appropriate.

Nimbus-LC Software Design

According to the release plan, one of the major goals of phase I is to create an architecture that will allow the bulk of the software to be independent of the hardware that it controls. Indeed, we want to separate the abstract behavior of the weather station from its concrete implementation.

For example, the software must be able to display the current temperature regardless of the hardware configuration. This implies the design shown in Figure 27-1.

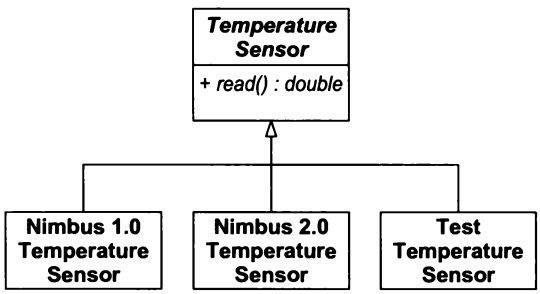


Figure 27-1 Initial Temperature-Sensor Design

An abstract base class named `TemperatureSensor` supplies a polymorphic `read()` function. Derivatives of this base class allow for separate implementations of the `read()` function.

The Test Classes. Notice that there is one derivative for each of the two known hardware platforms. There is also a special derivative named `TestTemperatureSensor`. This class will be used to test the software in a

1. We all know that the requirements document is the most volatile document in any software project.

workstation, which is not connected to Nimbus hardware. This allows the software engineers to write unit tests and acceptance tests for their software even when they don't have access to a Nimbus system.

Also, we have very little time to integrate the Nimbus 2.0 hardware and software together. The Nimbus 2.0 version will be at risk because of this short time frame. By making the Nimbus software work with both the Nimbus 1.0 hardware and with the test class, we will have made the Nimbus software execute on multiple platforms. This lessens the risk of significant portability issues with the Nimbus 2.0.

The test classes also give us the opportunity to test features or conditions that are hard to capture in the software. For example, we can set up the test classes to produce failures that are difficult to simulate with the hardware.

Making Periodic Measurements. The most common mode of the Nimbus system is when it is displaying current weather-monitoring data. Each of the values are updated at their own particular rate. Temperature is updated once per minute, while barometric pressure is updated once every five minutes. Clearly, we need some kind of scheduler that will trigger these readings and communicate them to the user. Figure 27-2 shows a possible structure.

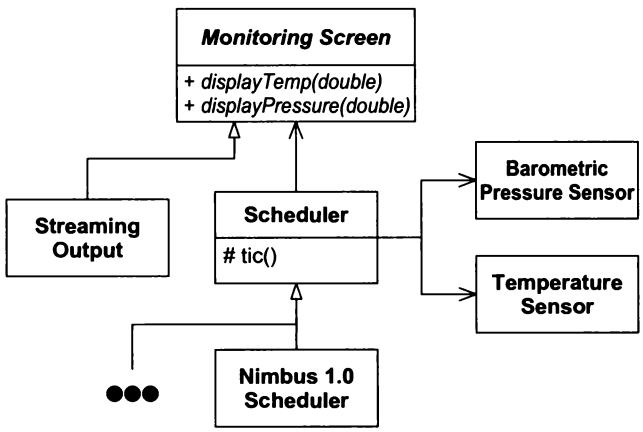


Figure 27-2 Initial Scheduler and Display architecture

We imagine the `Scheduler` to be a base class that has many possible implementations, one for each of the hardware and test platforms. The `Scheduler` has a `tic` function that it expects will be called once every 10 ms. It is the responsibility of the derived class to make this call. (See Figure 27-3.) The `Scheduler` counts the `tic()` calls. Once per minute, it calls the `read()` function of the `TemperatureSensor` and passes the returned temperature to the `MonitoringScreen`. For phase I, we don't need to show the temperature in a GUI, so the derivative of `MonitoringScreen` simply sends the result to an output stream.

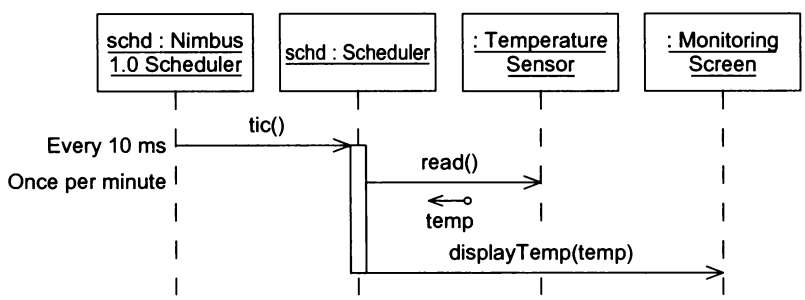


Figure 27-3 Initial Scheduler Sequence Diagram

Barometric Pressure Trend. The requirements document says that we must report the trend of the barometric pressure. This is a value that can have three states: *rising*, *falling*, or *stable*. How do we determine the value of this variable?

According to the Federal Meteorological Handbook,[2] the barometric-pressure trend is calculated as follows:

> *If the pressure is rising or falling at a rate of at least 0.06 inch per hour and the pressure change totals 0.02 inch or more at the time of the observation [to be taken once every three hours], a pressure change remark shall be reported.*

Where do we put this algorithm? If we put it in the `BarometricPressureSensor` class, then that class will need to know the time of each reading, and it will have to keep track of a series of readings going back three hours. Our current design does not allow for this. We could fix this by adding the current time as an argument to the `Read` function of the `BarometricPressureSensor` class and guaranteeing that that function will be called on a regular basis.

However, this couples the trend calculation to the frequency of user updates. It is not inconceivable that a change to the user interface update scheme could affect the pressure-trend algorithm. Also, it is very unfriendly for a sensor to demand that it be read on a regular basis in order to function properly. A better solution needs to be found.

We could have the `Scheduler` keep track of barometric pressure history and calculate trends at need. However, will we then also put temperature and wind speed history in the `Scheduler` class? Every new kind of sensor or history requirement would cause us to change the `Scheduler` class. This has the makings of a maintenance nightmare.

Reconsidering the `Scheduler`. Take another look at Figure 27-2. Notice that the `Scheduler` is connected to each of the sensors and to the user interface. As more sensors are added, and as more user interface screens are added, they will have to be added to the `Scheduler` too. Thus, the `Scheduler` is not closed to the addition of new sensors or user interfaces. This is a violation of the OCP. We would like to design the `Scheduler` so that it is independent of changes and additions to the sensors and user interfaces.

Decoupling the User Interface. User interfaces are volatile. They are subject to the whims of customers, marketing people, and nearly everyone else who comes in contact with the product. It seems very likely that if any part of the system suffers requirements thrashing, it will be the user interface. Therefore, we should decouple it first.

Figures 27-4 and 27-5 show a new design that uses the OBSERVER pattern. We have made the UI a dependent of the sensor, so that when the sensor reading changes, the UI will be automatically notified. Notice that the depen-

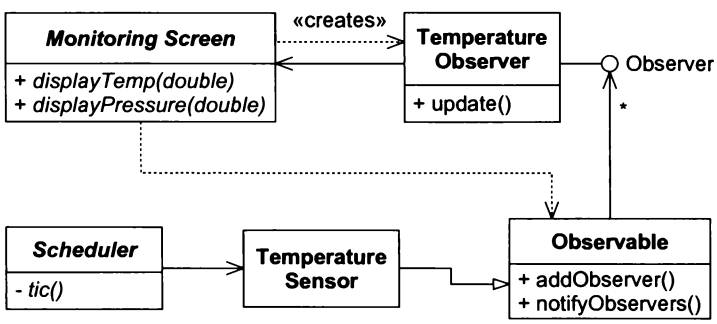


Figure 27-4 Observer decouples UI from Scheduler

2. *Federal Meteorological Handbook* No. 1, Chapter 11, Section 11.4.6 (http://www.nws.noaa.gov).

dency is indirect. The actual observer is an ADAPTER[3] named `TemperatureObserver`. This object is notified by the `TemperatureSensor` when the temperature reading changes. In response, the `TemperatureObsever` calls the `DisplayTemp` function of the `MonitoringScreen` object.

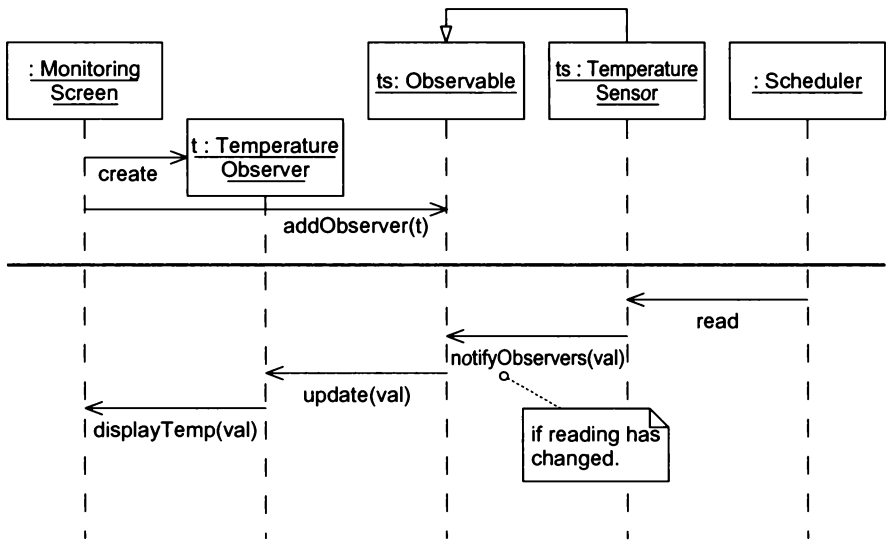


Figure 27-5 Decoupled UI sequence diagram

This design has nicely decoupled the UI from the `Scheduler`. The `Scheduler` now knows nothing of the UI and can focus solely upon telling the sensors when to read. The UI binds itself to the sensors and expects them to report any changes. However, the UI does not know about the sensors themselves. It simply knows about a set of objects that implements the `Observable` interface. This will allow us to add sensors without making significant changes to this part of the UI.

We have also solved the problem of the barometric-pressure trend. This reading can now be calculated by a separate `BarometricPressureTrendSensor` that observes the `BarometricPressureSensor`. (See Figure 27-6.)

Rethinking the Scheduler—Yet Again. The major role of the `Scheduler` is to tell each of the sensors when they should acquire a new value. However, if future requirements force us to add or remove a sensor, the `Scheduler` will need to be changed. Indeed, the `Scheduler` will have to change, even if we simply want to change the rate of a sensor. This is an unfortunate violation of the OCP. It seems that the knowledge of a sensor's polling rate belongs to the sensor itself and not any other part of the system.

We can decouple the `Scheduler` from the sensors by using the `Listener`[4] paradigm from the Java class library. This is similar to OBSERVER in that you register to be notified of something; but in this case, we want to be notified when a certain event (time) occurs. (See Figure 27-7.)

Sensors create anonymous ADAPTER classes that implement the `AlarmListener` interface. The sensors then register those adapters with the `AlarmClock` (the class we used to call the `Scheduler`). As part of the registration, they tell the `AlarmClock` how often they would like to be woken up (e.g., every second or every fifty milliseconds). When that period expires, the `AlarmClock` sends the `wakeup` message to the adapter, which then sends the `read` message to the sensor.

This has completely changed the nature of the `Scheduler` class. In Figure 27-2 it formed the center of our system and knew about most of the other components. But now it simply sits at the side of the system. It knows

3. [GOF95], p. 139.
4. [JAVA98], p. 360.

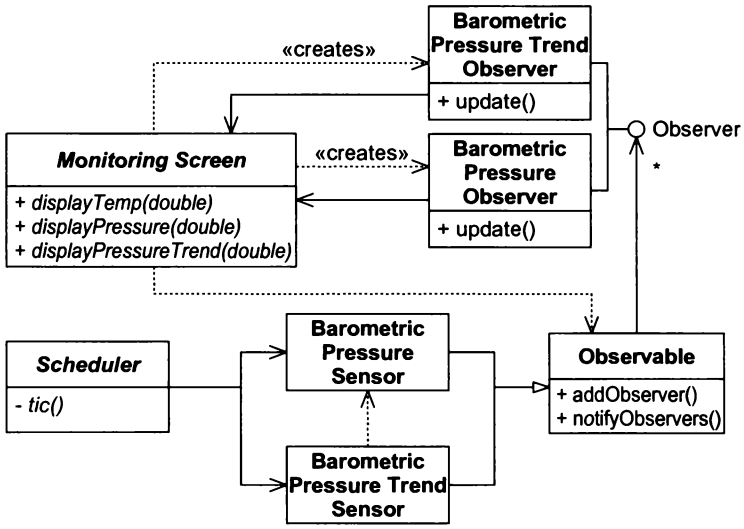


Figure 27-6 Barometric Pressure Observers

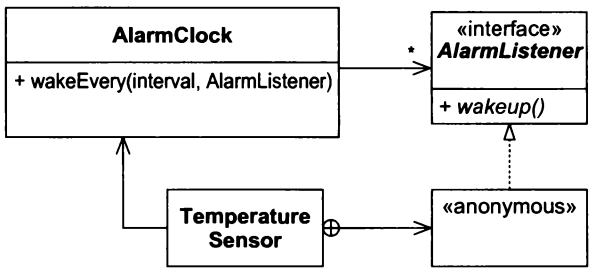


Figure 27-7 Decoupled Alarm clock

nothing about the other components. It conforms to the SRP by doing one job—scheduling—which has nothing whatever to do with weather monitoring. Indeed, it could be reused in many different kinds of applications. In fact, the change is so dramatic that we have changed the name to `AlarmClock`.

The Structure of the Sensors. Having decoupled the sensors from the rest of the system, we should look at their internal structure. Sensors now have three separate functions. First, they have to create and register the anonymous derivative of the `AlarmListener`. Second, they have to determine if their readings have changed and invoke the `notifyObservers` method of the `Observable` class. Third, they have to interact with the Nimbus hardware in order to read the appropriate values.

Figure 27-1 showed how these concerns might be separated. Figure 27-8 integrates that design with the other changes we have made. The `TemperatureSensor` base class deals with the first two concerns, since they are generic. The derivative of `TemperatureSensor` can then deal with the hardware and perform the actual readings.

Figure 27-8 employs TEMPLATE METHOD in order to achieve the separation between the generic and specific concerns of the `TemperatureSensor`. You can see this pattern in the private `check` and `read` functions of `TemperatureSensor`. When the `AlarmClock` calls `wakeup` on the anonymous class, the anonymous class forwards the call to the `check` function of the `TemperatureSensor`. The `check` function then calls the abstract `read` function of `TemperatureSensor`. This function will be implemented by the derivative to properly interact with the hardware and obtain the sensor reading. The `check` function then determines whether the new reading is different from the previous reading. If a difference is detected, then it notifies the waiting observers.

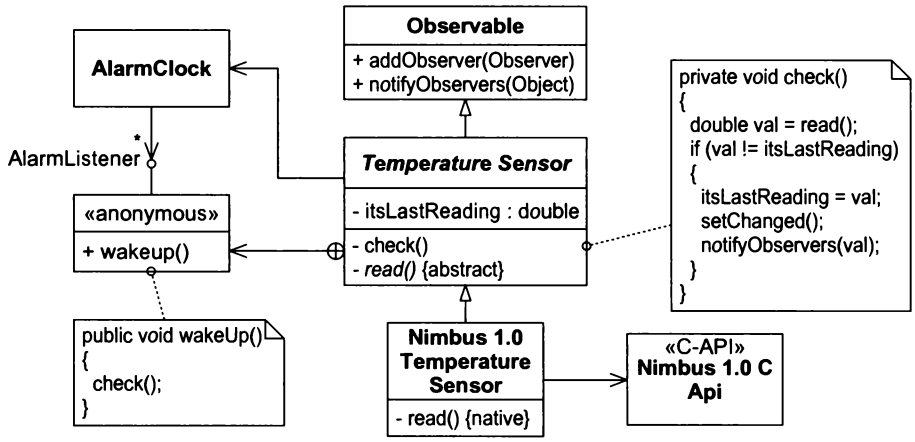


Figure 27-8 Sensor Structure

This nicely accomplishes the separation of concerns that we need. For every new hardware or testing platform, we will be able to create a derivative of `TemperatureSensor` that will work with it. Moreover, that derivative must merely override one very simple function: `read()`. The rest of the functionality of the sensor remains in the base class where it belongs.

Where Is the API? One of our Release II goals is the creation of a new API for the Nimbus 2.0 hardware. This API should be written in Java, be extensible, and provide simple and direct access to the Nimbus 2.0 hardware. Furthermore, it must serve the Nimbus 1.0 hardware as well. Without that API, all the simple debugging and calibration tools that we write for this project will have to be changed when the new board is introduced. Where is this API within our current design?

It turns out that nothing we have created so far can serve as a simple API. What we are looking for is something like this:

```
public interface TemperatureSensor
{
  public double read();
}
```

We are going to want to write tools that have direct access to this API without having to bother with registering observers. We also don't want sensors at this level to be polling themselves automatically, or interacting with the `AlarmClock`. We want something very simple and isolated that acts as the direct interface to the hardware.

It may seem that we are reversing all our previous arguments. After all, Figure 27-1 shows exactly what we have just asked for. However, the changes we made subsequent to Figure 27-1 were made for sound reasons. What we need is a hybrid that mixes the best of both schemes.

Figure 27-9 employs the BRIDGE pattern to extract the true API from the `TemperatureSensor`. The intent of this pattern is to separate an implementation from an abstraction, so that both may vary independently. In our case, the `TemperatureSensor` is the abstraction, and the `TemperatureSensorImp` is the implementation. Notice that the word "implementation" is being used to describe an abstract interface and that the "implementation" is itself implemented by the `Nimbus1.0TemperatureSensor` class.

Creational Issues. Look again at Figure 27-9. In order for this to work, a `TemperatureSensor` object must be created and bound to a `Nimbus1.0TemperatureSensor` object. Who takes care of this? Certainly, whatever part of the software is responsible for this will not be platform independent, since it must have explicit knowledge of the platform-dependent `Nimbus1.0TemperatureSensor`.

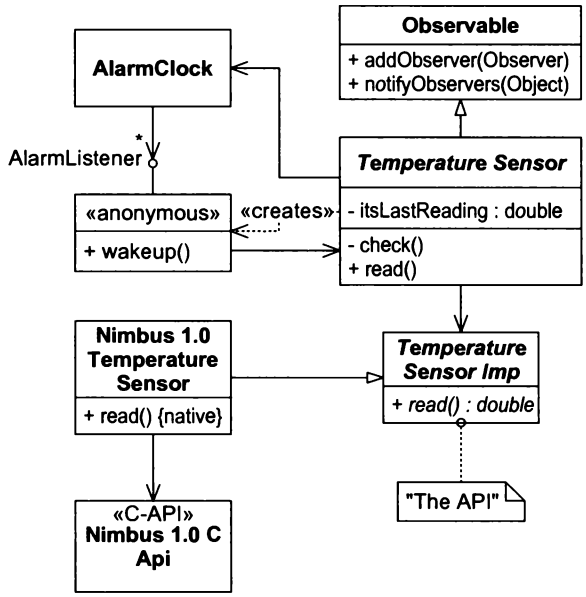


Figure 27-9 Temperature Sensor with API

We could use the main program to do all this. We could write it as shown in Listing 27-1.

Listing 27-1

`WeatherStation`

```
public class WeatherStation
{
  public static void main(String[] args)
  {
    AlarmClock ac = new AlarmClock(
      new Nimbus1_0AlarmClock;

    TemperatureSensor ts =
      new TemperatureSensor(ac,
       new Nimbus1_0TemperatureSensor);

    BarometricPressureSensor bps =
      new BarometricPressureSensor(ac,
        new Nimbus1_0BarometricPressureSensor);

    BarometricPressureTrend bpt =
      new BarometricPressureTrend(bps)
  }
}
```

This is a workable solution, but requires an awful lot of clerical overhead. Instead, we could use FACTORIES to deal with most of the clerical overhead involved with creation. Figure 27-10 shows the structure.

We have named the factory the `StationToolkit`. This is an interface that presents methods that offer to create instances of the API classes. Each platform will have its own derivative of `StationToolkit`, and that derivative will create the appropriate derivatives of the API classes.

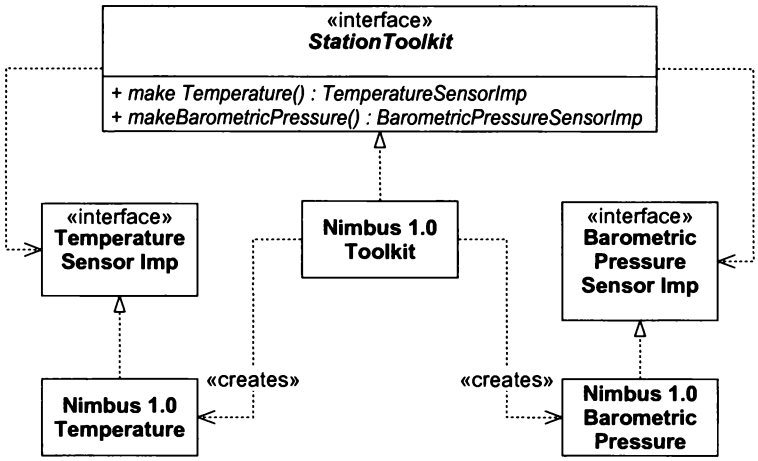


Figure 27-10 Station Toolkit

Now we can rewrite the main function as shown in Listing 27-2. Notice that in order to alter this main program to work with a different platform, all we have to change is the two lines that create the `Nimbus1.0AlarmClock` and the `Nimbus1.0Toolkit`. This is a dramatic improvement over Listing 27-1, which required a change for every sensor it created.

Listing 27-2

`WeatherStation`

```
public class WeatherStation
{
  public static void main(String[] args)
  {
    AlarmClock ac = new AlarmClock(
      new Nimbus1_0AlarmClock;

    StationToolkit st = new Nimbus1_0Toolkit();

    TemperatureSensor ts =
      new TemperatureSensor(ac,st);

    BarometricPressureSensor bps =
      new BarometricPressureSensor(ac,st);

    BarometricPressureTrend bpt =
      new BarometricPressureTrend(bps)
  }
}
```

Notice that the `StationToolkit` is being passed into each sensor. This allows the sensors to create their own implementations. Listing 27-3 shows the constructor for `TemperatureSensor`.

Listing 27-3

`TemperatureSensor`

```
public class TemperatureSensor extends Observable
{
  public TemperatureSensor(AlarmClock ac,
                           StationToolkit st)
```

```
  {
    itsImp = st.makeTemperature();
  }
  private TemperatureSensorImp itsImp;
}
```

Getting the Station Toolkit to Create the AlarmClock. We can improve matters further by having the `StationToolkit` create the appropriate derivative of the `AlarmClock`. Once again, we will employ the BRIDGE pattern to separate the `AlarmClock` abstraction that is meaningful to the weather-monitoring applications, from the implementation that supports the hardware platform.

Figure 27-11 shows the new `AlarmClock` structure. The `AlarmClock` now receives `tic()` messages through its `ClockListener` interface. These messages are sent from the appropriate derivative of the `AlarmClockImp` class in the API.

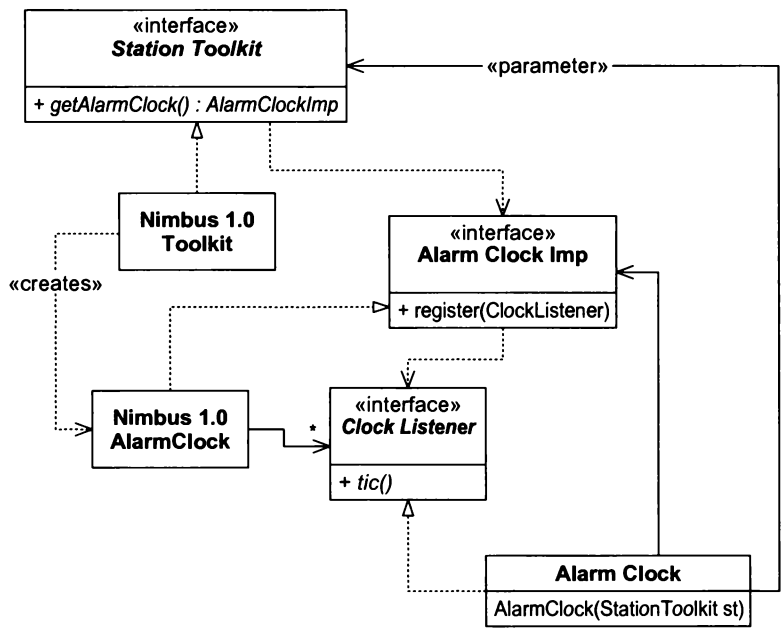


Figure 27-11 Station Toolkit and Alarm Clock

Figure 27-12 shows how the `AlarmClock` gets created. The appropriate `StationToolkit` derivative is passed into the constructor of the `AlarmClock`. The `AlarmClock` directs it to create the appropriate derivative of `AlarmClockImp`. This is passed back to the `AlarmClock`, and the `AlarmClock` registers with it so that it will receive `tic()` messages from it.

Once again, this has an effect upon the main program in Listing 27-4. Notice that now there is only one line that is platform dependent. Change that line, and the entire system will use a different platform.

Listing 27-4

`WeatherStation`

```
public class WeatherStation
{
  public static void main(String[] args)
  {
    StationToolkit st = new Nimbus1_0Toolkit();
```